Action-Prefix System — Test Plan (all supported test types)

Field	Value
Revision	1
Created	2026-06-09
Last modified	2026-06-09T00:00:00Z
Status	active
Scope	unit + integration + e2e + meta-test (paired §1.1 mutation) for §11.4.140
Inputs	`RESEARCH.md`, `DESIGN.md`, `RULE_DRAFT.md`

> Anti-bluff (§11.4 / §11.4.27 / §11.4.107): every test below produces a captured-evidence artefact under `qa-results/action_prefix/<run-id>/`. A PASS without its cited evidence file is a §11.4 PASS-bluff. No mock outside unit tests (§11.4.27): integration + e2e drive the REAL registry + REAL hook + a REAL agent invocation.

---

1. UNIT — registry parse + grammar match/no-match/escape (the expander)

Target: `scripts/actions/expand_prefix.sh` (the pure expander) + the registry. Driver: a bash test harness `test_action_prefix_unit.sh` (sh -n + bash -n clean per §11.4.67), N=3 deterministic per §11.4.50.

#	Case	Input prompt (first line)	Expect	Evidence
U1	registry parses	—	YAML loads; schema_version present; BACKGROUND present	`registry_pars`
U2	exact match	`BACKGROUND :: do X`	matched=true, action=BACKGROUND,	`match_backgro`

<table>
<tr><th>#</th><th>Case</th><th>Input prompt (first line)</th><th>Expect</th><th>Evidence</th></tr>
<tr><td></td><td></td><td></td><td>residual=<code>do X</code>, expansion==registry verbatim</td><td></td></tr>
<tr><td>U3</td><td>lowercase no-match</td><td><code>background :: do X</code></td><td>matched=false (ordinary prompt)</td><td><code>nomatch_lower</code></td></tr>
<tr><td>U4</td><td>mid-prose no-match</td><td><code>please BACKGROUND :: do X</code></td><td>matched=false</td><td><code>nomatch_midpr</code></td></tr>
<tr><td>U5</td><td>wrong separator no-match</td><td><code>BACKGROUND::do X</code> / <code>BACKGROUND: do X</code> / <code>Foo::Bar</code></td><td>matched=false (no <code>" :: "</code>)</td><td><code>nomatch_separ</code></td></tr>
<tr><td>U6</td><td>escape</td><td><code>\BACKGROUND :: do X</code></td><td>matched=false; emitted prompt=<code>BACKGROUND :: do X</code> (backslash stripped, NO expansion)</td><td><code>escape_litera</code></td></tr>
<tr><td>U7</td><td>stacked prefixes</td><td><code>OUTER :: BACKGROUND :: do X</code> (with a second test action OUTER)</td><td>both expand outer-to-inner; residual=<code>do X</code></td><td><code>stacked.json</code></td></tr>
<tr><td>U8</td><td>unknown grammar-shaped token</td><td><code>BACGROUND :: do X</code> (typo)</td><td>matched=false-but-shaped; verdict=ASK (names closest registered action); NO invented expansion</td><td><code>unknown_ask.j</code></td></tr>
<tr><td>U9</td><td>empty / blank prompt</td><td>`<code>/ whitespace | matched=false, no crash |</code>empty.json<code>| | U10 | leading blank lines |</code>:: do X<code>| matched=true (first NON-blank line) |</code>leading_blanks.json<code>| | U11 | BACKGROUND expansion fidelity |</code>BACKGROUND :: x<code>| emitted expansion byte-equals the operator's verbatim text |</code>expansion_fidelity.json`</td><td></td><td></td></tr>
</table>

PASS criterion: every case’s actual == expected across N=3 identical runs (§11.4.50 evidence-hash identical). Each row writes its JSON evidence; the harness asserts presence + content via `ab_pass_with_evidence` (§11.4.69).

---

2. INTEGRATION — the Claude `UserPromptSubmit` hook rewrites correctly

Target: `scripts/hooks/action-prefix-expand.sh` (the real hook, reading the real registry). No mock (§11.4.27). Driver: `test_action_prefix_hook.sh`.

#	Case	Feed on stdin (UserPromptSubmit JSON)	Expect on stdout
I1	match → additionalContext	`{"prompt":"BACKGROUND :: build the parser"}`	exit 0 + JSON `hookSpecificOutput.additionalC` containing the BACKGROUND expansion + residual
I2	no-match → no-op	`{"prompt":"build the parser"}`	exit 0 + empty stdout (no injec
I3	escape → no-op	`{"prompt":"\\BACKGROUND :: x"}`	exit 0 + empty (literal)
I4	unknown shaped → clarify note	`{"prompt":"BACGROUND :: x"}`	exit 0 + additionalContext ask which action (§11.4.66/§11.4.1 no invented expansion
I5	no-jq fallback parity	I1 with `PATH` stripped of `jq`	identical output to I1 (embedd extractor)
I6	registry path override	`HELIX_ACTION_REGISTRY=/tmp/alt.yaml` with a custom action	hook expands the custom actic (decoupling proof, §11.4.28)

PASS criterion: stdout JSON validates + `additionalContext` contains the registry-verbatim expansion (I1/I5/I6); empty for no-op cases. Hook contract matches RESEARCH.md §1.1 (additive `additionalContext`, exit 0).

---

3. E2E — a real agent given a `BACKGROUND :: …` prompt applies the expansion

Per §11.4.52 (autonomous) + §11.4.98 (no manual intervention) + §11.4.107 (real-content liveness). Two tiers honour the §11.4.3 topology split:

- **E3a — Claude Code (LAYER 2 mechanical, autonomous):** run Claude Code headless (`claude -p`) with the hook installed, feeding `BACKGROUND :: print the words HELIX_BG_OK to a file`. Assert (i) the agent's transcript shows the BACKGROUND expansion was applied (it announces/uses the background-

subagent + captured-evidence framing), and (ii) the residual task ran. Captured evidence: the session transcript under `docs/qa/<run-id>/e3a_claude/` + the produced file. This is the autonomous, re-runnable path (`-count=3` per §11.4.98).
- **E3b — non-hook agent (LAYER 1, self-applied):** with the §11.4.140 mirror block present in the agent's carrier (AGENTS.md/GEMINI.md/QWEN.md), feed the same prompt to a non-Claude agent (Gemini CLI or Codex CLI, whichever is installed) and assert the agent's response reflects the expansion (it treats the work as background/subagent + evidence-backed). Captured: transcript under `docs/qa/<run-id>/e3b_<agent>/`. Where no second agent is installed → SKIP-with-reason `hardware_not_present/feature_disabled_by_config` per §11.4.69 + a tracked operator-attended migration item (§11.4.52) — NEVER a fake PASS.

Honest §11.4.6 boundary: E3b asserts the model *applies the expansion's intent*, which is a judgement-bearing assertion. To keep it anti-bluff, the unknown/escape/no-match control prompts (E3c) MUST produce the OPPOSITE behaviour (no background framing) — the metamorphic relation (§11.4.107(8)) proving the expansion is what caused the behaviour, not a generic prior.

---

4. META-TEST — paired §1.1 mutation (the gate is not a bluff gate)

Target gate: `CM-ACTION-PREFIX-SYSTEM`. The gate asserts: registry exists + parses; `prefix_regex` present; the BACKGROUND `expansion` byte-matches the operator verbatim; the LAYER-1 mirror block present in all four carriers (CLAUDE.md/AGENTS.md/QWEN.md/GEMINI.md); the hook + expander scripts exist + executable + `sh -n`/`bash -n` clean.

Paired mutations (each MUST flip the gate to FAIL, then restore to PASS):

M	Mutation	Gate must
M1	corrupt one word of the BACKGROUND `expansion` in `registry.yaml`	FAIL (`expansion` fidelity)
M2	delete `grammar.prefix_regex`	FAIL (grammar missing)
M3	remove the §11.4.140 mirror block from `GEMINI.md`	FAIL (carrier coverage incomplete)

M	Mutation	Gate must
M4	`chmod -x scripts/hooks/action-prefix-expand.sh`	FAIL (hook not executable)
M5	strip the literal `11.4.140` from one consumer file	`CM-COVENANT-114-140-PROPAGATION` FAILs

Plus a behavioural mutation proving the EXPANDER catches its own negation: make `expand_prefix.sh` always return matched=false → unit cases U2/U7/U11 FAIL. And make it expand on lowercase → U3 FAILs. Restore after each (§11.4.84 quiescence — mutations serialised, working tree clean before any commit).

---

5. Evidence + determinism summary

- Every unit/integration/e2e case writes a captured-evidence artefact under `qa-results/action_prefix/<run-id>/` (unit/integration) or `docs/qa/<run-id>/` (e2e), cited by the PASS line via `ab_pass_with_evidence` (§11.4.69).
- All harnesses are `sh -n` + `bash -n` clean (§11.4.67), run N=3 with identical exit codes + evidence-hashes (§11.4.50), and self-clean (§11.4.14, trap EXIT).
- The e2e tier is fully automated + re-runnable `-count=3` (§11.4.98); the non-Claude tier SKIPs honestly with a tracked migration item when no second agent is installed (§11.4.3 / §11.4.52) — never a fake PASS.
- This four-type coverage (unit + integration + e2e + meta-test mutation) is the §11.4.4(b) four-layer floor for §11.4.140; the HelixQA Challenge-bank entry (a `CME-ACTION-PREFIX-001` Challenge dispatching to E3a/E3b) is the user-visible layer 4 (§11.4.27).